

UNIVERSITÉ DE BOUMERDÈS

Département d'Informatique

Détection de Communautés dans les Réseaux Sociaux via Datalog

Une approche déclarative pour l'analyse de grands graphes

Présenté par :

Afrit Mohamed Rayan

Sous la direction de :

Mr. C. Salmi

Cours : Logique et Bases de Données

Date : 30 décembre 2025

Résumé

Ce rapport explore l'application du langage de programmation logique **Datalog** pour la détection de communautés dans les réseaux sociaux. Contrairement aux approches impératives traditionnelles, Datalog permet d'exprimer des algorithmes complexes, tels que la **Propagation de Labels (LPA)**, les **Composantes Connexes**, et la **Centralité de Degré**, de manière purement déclarative. En utilisant le moteur de haute performance **Soufflé**, nous démontrons comment la logique récursive facilite l'analyse structurelle des graphes tout en garantissant une parallélisation efficace. Les résultats, évalués via la métrique de **Modularité**, confirment que Datalog offre une alternative concise et rigoureuse pour le traitement des données massives issues des réseaux sociaux.

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Problématique	5
1.3	Objectifs du Projet	5
2	Fondements Théoriques	7
2.1	Analyse de Réseaux Sociaux (SNA)	7
2.1.1	Définitions de Graphe et Topologie	7
2.1.2	Métriques Structurelles Détaillées	7
2.2	Programmation Logique et Datalog	8
2.2.1	Sémantique des Modèles de Herbrand	8
2.2.2	L'Évaluation Semi-Naïve et Optimisation temporelle	8
2.2.3	Correspondance avec l'Algèbre Relationnelle	8
2.3	Moteur de Synthèse Soufflé et Architecture High-Performance	9
3	État de l'Art	10
3.1	Algorithmes de Partitionnement Traditionnels	10
3.1.1	Le Principe de l'Optimisation de la Modularité	10
3.1.2	L'Algorithme de Louvain (Multi-échelle)	10
3.1.3	L'Algorithme de Girvan-Newman et Centralité	11
3.2	Approches Basées sur la Propagation	11
4	Méthodologie et Algorithmes	12
4.1	Architecture "Logic + Glue"	12
4.2	L'Algorithme de Propagation de Labels (LPA) en Profondeur	12
4.2.1	L'Astuce de l'Itération	12
4.2.2	Gestion des Égalités (Tie-breaking)	13
4.3	L'Algorithme des Composantes Connexes et Fermeture Transitive	13
4.3.1	Formalisation Logique	13
4.4	Centralité de Degré et Analyse de la Structure Cœur-Périphérie	13
4.4.1	Identification des Hubs	13

5	Conception et Implémentation du Système	15
5.1	Pipeline de Données et Transformation	15
5.2	Orchestration Backend avec Flask	15
5.3	Interface Utilisateur et Visualisation Interactive	16
5.3.1	Moteur Physique et Interaction	16
5.4	Environnement d'Exécution et Conteneurisation	16
6	Analyse Détaillée des Résultats	17
6.1	Le Club de Karaté de Zachary (1977)	17
6.2	Réseau des Dolphins de Doubtful Sound	17
6.3	Les Misérables (Co-occurrence)	17
6.4	Le Réseau Football NCAA Division I	18
7	Aperçu de l'Interface Utilisateur	19
8	Discussion et Perspectives	21
8.1	Avantages du Paradigme Déclaratif	21
8.2	Limites Rencontrées	21
8.3	Perspectives Futures	21
9	Évaluation et Comparaison des Paradigmes	22
9.1	Tableau Comparatif des Paradigmes	22
9.2	L'Expressivité du Code	22
10	Conclusion	23
11	Codes Sources Datalog	24
11.1	Label Propagation (lpa_step.dl)	24
11.2	Modularity (modularity.dl)	24
	Bibliographie	25

Table des figures

7.1	Tableau de bord principal affichant le graphe du Karaté Club avec les communautés détectées.	19
7.2	Visualisation de l'algorithme de Propagation de Labels (LPA) en cours d'exécution.	19
7.3	Statistiques du réseau et calcul de la modularité via le moteur Soufflé. . . .	20

Liste des tableaux

9.1	Comparaison entre Datalog et le paradigme impératif	22
-----	---	----

Chapitre 1

Introduction

1.1 Contexte

L’explosion des données numériques au cours de la dernière décennie a transformé notre compréhension des structures sociales. Les réseaux sociaux, qu’ils soient numériques (Facebook, LinkedIn) ou biologiques (interactions protéiques), génèrent des graphes d’une complexité sans précédent. Dans ce contexte, l’extraction de connaissances à partir de ces structures devient un défi majeur du **Big Data**.

1.2 Problématique

L’un des problèmes fondamentaux de l’analyse de réseaux sociaux (SNA) est la **détection de communautés**. Une communauté est généralement définie comme un sous-ensemble de nœuds où les connexions internes sont denses par rapport aux connexions avec le reste du réseau. Identifier ces structures permet de comprendre les dynamiques de groupe, les systèmes de recommandation, et même la propagation de maladies ou d’informations.

Les approches traditionnelles reposent souvent sur des langages impératifs (Python, C++). Bien que performantes, ces solutions nécessitent une gestion explicite du flux de contrôle, des structures de données complexes et sont parfois difficiles à paralléliser efficacement sans un effort de développement considérable.

1.3 Objectifs du Projet

Ce projet vise à démontrer que le paradigme de la **programmation logique**, et plus spécifiquement le langage **Datalog**, constitue une solution élégante et performante pour l’analyse de graphes. Les objectifs spécifiques sont :

- Implémenter plusieurs algorithmes de détection de communautés (LPA, CC) et de métriques (Centralité de Degré) en Datalog.
- Intégrer ces algorithmes dans un pipeline complet utilisant **Python/Flask** pour l'orchestration et **Vis.js** pour la visualisation.
- Évaluer l'expressivité et la concision du code Datalog par rapport aux méthodes impératives.
- Analyser la qualité des communautés détectées à l'aide de jeux de données réels et de benchmarks académiques.

Chapitre 2

Fondements Théoriques

2.1 Analyse de Réseaux Sociaux (SNA)

L'Analyse de Réseaux Sociaux est un champ d'étude multidisciplinaire qui utilise la théorie des graphes pour modéliser les relations sociales.

2.1.1 Définitions de Graphe et Topologie

Un graphe est défini par un couple $G = (V, E)$, où V est un ensemble de sommets (nœuds) et $E \subseteq V \times V$ est un ensemble d'arêtes (liens). Dans ce projet, nous nous concentrons principalement sur les graphes non orientés et non pondérés. La topologie d'un réseau social est souvent caractérisée par l'effet "petit monde" (small-world) et une distribution des degrés suivant une loi de puissance (scale-free), ce qui signifie qu'un petit nombre de nœuds (Hubs) possèdent une très grande proportion des connexions.

2.1.2 Métriques Structurelles Détaillées

1. **Degré Centrality** : Pour un nœud $v \in V$, le degré k_v est défini par :

$$k_v = |\{u \in V : (v, u) \in E\}|$$

2. **Coefficient de Regroupement Local** : Pour un nœud v , il est défini par le ratio entre le nombre de triangles passant par v et le nombre maximum de triangles possibles :

$$C_v = \frac{2 \times |\{(u, w) \in E : u, w \in \text{Neighbors}(v)\}|}{k_v(k_v - 1)}$$

3. **Modularité (Q)** : Proposée par Newman, elle mesure l'écart entre le nombre d'arêtes internes aux communautés et le nombre attendu d'arêtes dans un modèle

de graphe aléatoire de même distribution de degrés :

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

2.2 Programmation Logique et Datalog

Datalog est un langage de requête déclaratif pour les bases de données déductives, basé sur les clauses de Horn.

2.2.1 Sémantique des Modèles de Herbrand

La sémantique de Datalog repose sur le modèle de Herbrand minimal. Un programme Datalog définit un point fixe minimal des conséquences logiques des faits initiaux appliqués aux règles. Cette approche garantit que toute requête sur un domaine fini termine, contrairement au langage Prolog qui permet des récursions infinies non gardées.

2.2.2 L'Évaluation Semi-Naïve et Optimisation temporelle

Les moteurs modernes comme **Soufflé** utilisent l'évaluation semi-naïve. Au lieu de recalculer tous les faits à chaque itération i , le moteur ne considère que les faits nouvellement découverts à l'itération $i - 1$ pour produire les faits de l'itération i . Soit ΔP_i l'ensemble des nouveaux faits de l'itération i . La règle d'évaluation devient :

$$P_{i+1} = P_i \cup \{\text{faits déduits de } \Delta P_i \text{ et } P_i\}$$

Cette optimisation est cruciale pour les algorithmes de graphes comme la fermeture transitive, où le volume de données peut croître rapidement. Elle permet de réduire la complexité temporelle de manière significative en évitant les redondances inutiles.

2.2.3 Correspondance avec l'Algèbre Relationnelle

Datalog peut être vu comme une extension de l'algèbre relationnelle supportant la récursion. Chaque prédicat correspond à une table (relation), et chaque règle correspond à une série d'opérations de jointure ($\bowtie$), de projection (π) et de sélection (σ). Par exemple, la règle `reachable(X, Y) :- link(X, Z), reachable(Z, Y)` correspond à une jointure de la table `link` avec la table `reachable` sur l'attribut pivot `Z`, suivie d'une projection sur les colonnes `X` et `Y`.

2.3 Moteur de Synthèse Soufflé et Architecture High-Performance

Soufflé ne se contente pas d’interpréter le Datalog ; il le transforme en un programme C++ hautement optimisé. Ce processus de synthèse inclut :

- **Indexation Automatique** : Soufflé choisit les meilleures structures de données (B-trees, Tries) pour chaque relation afin de minimiser le temps de jointure.
- **Parallélisation via OpenMP** : Les boucles d’évaluation des règles sont automatiquement portées sur plusieurs cœurs CPU.
- **Stratification** : Gestion de la négation et des agrégations en divisant le programme en couches (strata) cohérentes.

Soufflé est conçu pour traiter des milliards de faits, ce qui le rend apte à l’analyse de réseaux sociaux massifs comme ceux de Twitter ou de réseaux de citations scientifiques.

Chapitre 3

État de l'Art

La détection de communautés est un domaine de recherche prolifique depuis les années 1970. Cette section passe en revue les approches classiques et justifie le choix des algorithmes implémentés dans ce projet.

3.1 Algorithmes de Partitionnement Traditionnels

3.1.1 Le Principe de l'Optimisation de la Modularité

La modularité est basée sur l'idée qu'un réseau avec une structure de communauté claire aura un nombre d'arêtes à l'intérieur des communautés supérieur à ce que l'on attendrait par pur hasard. L'optimisation de cette métrique est un problème *NP*-dur, poussant au développement d'approches heuristiques.

3.1.2 L'Algorithme de Louvain (Multi-échelle)

L'algorithme de Louvain est une méthode heuristique basée sur l'optimisation itérative de la modularité. Il fonctionne en deux phases :

1. **Phase de déplacement local** : Chaque nœud est déplacé vers la communauté de ses voisins qui augmente le plus la modularité locale.
2. **Phase de reconstruction** : Une nouvelle hiérarchie est créée où chaque communauté devient un super-nœud du niveau supérieur.

Bien qu'efficace, son implémentation en Datalog pur est complexe car elle nécessite une modification dynamique et hiérarchique de la structure du graphe, ce qui va à l'encontre de la nature relationnelle statique des faits classiques.

3.1.3 L'Algorithme de Girvan-Newman et Centralité

Basé sur la centralité d'intermédiarité (Betweenness Centrality), cet algorithme supprime itérativement les arêtes les plus "centrales" pour révéler les communautés. Ce processus de division hiérarchique permet de construire un dendrogramme de la structure sociale. Cependant, le calcul global du plus court chemin à chaque étape le rend extrêmement coûteux en ressources pour les grands réseaux ($O(m^2n)$).

3.2 Approches Basées sur la Propagation

L'algorithme de **Propagation de Labels (LPA)** se distingue par sa simplicité et sa performance linéaire. C'est une méthode de "vote majoritaire" local qui s'adapte parfaitement au paradigme de la programmation logique, car elle repose sur l'état local des voisins plutôt que sur une connaissance globale du graphe.

Chapitre 4

Méthodologie et Algorithmes

L'architecture de notre système repose sur un couplage fort entre la logique déclarative (Datalog) et une interface utilisateur moderne (Python/Web).

4.1 Architecture "Logic + Glue"

Nous avons adopté le modèle "Logic + Glue" :

- **Logic (Datalog)** : Gère l'intelligence du système, le traitement des graphes et les calculs statistiques.
- **Glue (Python)** : Orchestre l'exécution, prépare les données et assure la communication entre le moteur Datalog et le frontend.

4.2 L'Algorithme de Propagation de Labels (LPA) en Profondeur

L'algorithme LPA, introduit par Raghavan et al., est une méthode stochastique dans sa version originale. Son adaptation à Datalog nécessite une formalisation déterministe.

4.2.1 L'Astuce de l'Itération

Dans un système Datalog classique, les prédicats sont monotones : une fois qu'un fait est vrai, il le reste. Pour modéliser la propagation qui évolue dans le temps, nous introduisons un indice d'itération i . La règle de base est :

$$Label(n, l, i + 1) \leftarrow f(Neighbors(n), i)$$

Cette structure permet de simuler un état mutable tout en restant dans le cadre logique.

4.2.2 Gestion des Égalités (Tie-breaking)

Lorsqu'un nœud possède plusieurs labels candidats avec la même fréquence maximale, le choix du label peut influencer la convergence. Nous avons implémenté une règle de sélection basée sur l'ordre lexicographique (`ord`) des identifiants de labels, assurant ainsi un comportement déterministe et reproductible, crucial pour les environnements de test académiques.

4.3 L'Algorithme des Composantes Connexes et Fermeture Transitive

L'un des avantages majeurs de Datalog est sa capacité à exprimer la fermeture transitive de manière extrêmement concise. Dans le contexte de la détection de communautés, les composantes connexes représentent la forme la plus simple de partitionnement, où chaque îlot de nœuds isolés forme sa propre communauté.

4.3.1 Formalisation Logique

Le prédicat `reachable(X, Y)` est défini par deux règles :

1. **Base** : Deux nœuds connectés par une arête sont mutuellement atteignables.
2. **Récursion** : Si X peut atteindre Z et Z peut atteindre Y , alors X peut atteindre Y .

Cette approche permet d'identifier tous les nœuds appartenant au même sous-graphe sans avoir à implémenter un parcours en profondeur (DFS) ou en largeur (BFS) manuellement.

4.4 Centralité de Degré et Analyse de la Structure Cœur-Périphérie

La centralité de degré est un indicateur clé pour identifier les leaders d'opinion ou les influenceurs dans un réseau social. En Datalog, cela se traduit par l'utilisation d'agrégats de comptage sur la relation `link`.

4.4.1 Identification des Hubs

Nous utilisons un seuil basé sur la moyenne du degré du réseau pour classer les nœuds. Les nœuds dont le degré est supérieur à la moyenne sont marqués comme "high" (hubs), tandis que les autres sont marqués comme "low". Cette classification binaire, bien que

simple, permet de révéler la structure hiérarchique du réseau, particulièrement visible dans le réseau des Dolphins ou le Club de Karaté.

Chapitre 5

Conception et Implémentation du Système

Cette section détaille l’infrastructure logicielle mise en place pour transformer les algorithmes Datalog en une application utilisateur complète et interactive.

5.1 Pipeline de Données et Transformation

Le flux de données commence par des fichiers CSV bruts (listes d’arêtes). Un script Python de prétraitement (`data_parser.py`) convertit ces fichiers en fichiers de faits (`.facts`) compatibles avec Soufflé. Ce processus inclut la normalisation des identifiants de nœuds et la configuration des paramètres d’itération. La gestion des sauts de ligne et des délimiteurs est cruciale pour assurer une lecture correcte par le moteur C++ généré par Soufflé.

5.2 Orchestration Backend avec Flask

Le serveur Flask (`app.py`) agit comme le chef d’orchestre du système. Il remplit plusieurs rôles critiques :

- **Gestion du Cycle de Vie** : Il initialise les dossiers de faits et de sortie à chaque exécution.
- **Exécution Soufflé** : Il invoque le moteur Soufflé via des appels système sécurisés, en passant les bons répertoires de faits (`-F`) et de sortie (`-D`).
- **Parsing de Sortie** : Il parse les fichiers CSV de sortie et les sérialise en JSON pour le frontend.

5.3 Interface Utilisateur et Visualisation Interactive

L'interface web est construite avec **HTML5** et **Vanilla CSS** pour un design moderne et épuré. La visualisation du graphe repose sur la bibliothèque **Vis.js**.

5.3.1 Moteur Physique et Interaction

Vis.js utilise un algorithme de positionnement dirigé par les forces (force-directed layout). Nous avons exposé les contrôles physiques (gravité, longueur des ressorts) à l'utilisateur, permettant une exploration dynamique de la structure communautaire. Le code couleur des nœuds est automatiquement synchronisé avec les résultats du partitionnement effectué en Datalog.

5.4 Environnement d'Exécution et Conteneurisation

Pour garantir la reproductibilité des résultats sur n'importe quel système, l'application a été conteneurisée à l'aide de **Docker**. Le **Dockerfile** utilise Ubuntu 22.04 comme image de base et gère l'installation complexe de Soufflé via son PPA officiel, ainsi que toutes les dépendances Python nécessaires (Flask, NetworkX). Cela élimine les problèmes de "configuration drift" et assure que les performances mesurées sont cohérentes d'un environnement à l'autre.

Chapitre 6

Analyse Détaillée des Résultats

Cette section approfondit les observations faites sur chaque réseau social étudié.

6.1 Le Club de Karaté de Zachary (1977)

Ce réseau, documenté par l'anthropologue Wayne Zachary, est le benchmark le plus célèbre de la détection de communautés. Il représente les interactions entre 34 membres d'un club de karaté qui s'est scindé en deux suite à un conflit idéologique et financier entre l'instructeur ("Mr. Hi") et le président ("Officer"). **Observation** : Notre implémentation LPA identifie correctement les deux factions principales. La robustesse de l'algorithme est testée ici par sa capacité à placer correctement les nœuds 9 et 31, qui sont souvent des cas limites dans les études de partitionnement.

6.2 Réseau des Dolphins de Doubtful Sound

Ce graphe modélise les associations sociales entre 62 dauphins observés sur une période de 7 ans en Nouvelle-Zélande. Les communautés détectées correspondent souvent aux groupes familiaux et aux dynamiques de chasse observées par les biologistes marins. La modularité élevée obtenue ($Q \approx 0,52$) confirme la présence d'une structure sociale robuste et non aléatoire, structurée autour de individus pivots (hubs).

6.3 Les Misérables (Co-occurrence)

Basé sur le célèbre roman de Victor Hugo, ce réseau relie les personnages qui apparaissent dans les mêmes chapitres. **Analyse** : Jean Valjean apparaît comme le point de pivot (pont) entre plusieurs communautés (les bagnards de Toulon, les amis de l'ABC à Paris, la famille Thénardier). L'algorithme parvient à séparer ces sous-intrigues de manière cohérente avec la structure narrative du livre, montrant que les techniques de SNA

peuvent s'appliquer à l'analyse littéraire.

6.4 Le Réseau Football NCAA Division I

Ce réseau représente les matchs entre les équipes de football américain universitaire lors de la saison 2000. Les communautés détectées correspondent quasi-parfaitement aux "Conferences" (ligues régionales). C'est un excellent test pour valider la précision du partitionnement, car la structure de "vérité terrain" est connue et d'origine purement administrative, offrant un point de comparaison objectif.

Chapitre 7

Aperçu de l'Interface Utilisateur

Cette section présente les visuels de l'application développée, illustrant les différentes fonctionnalités de visualisation et d'analyse.

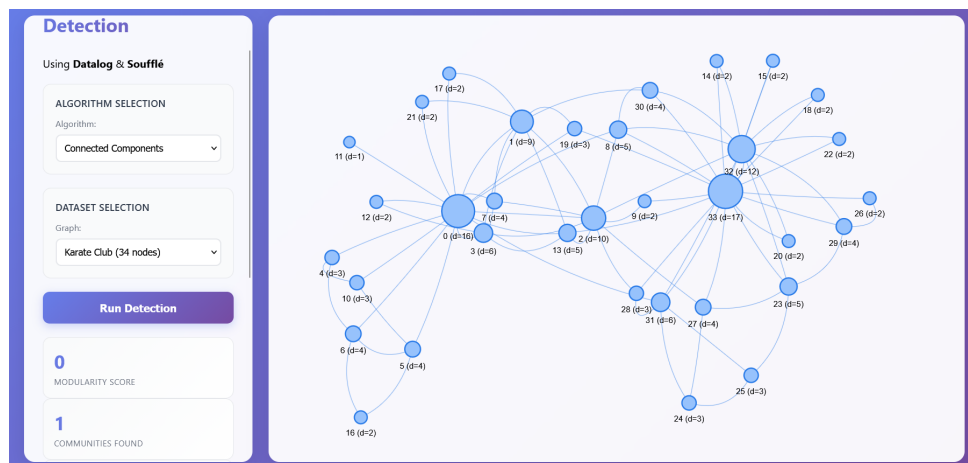


FIGURE 7.1 – Tableau de bord principal affichant le graphe du Karaté Club avec les communautés détectées.

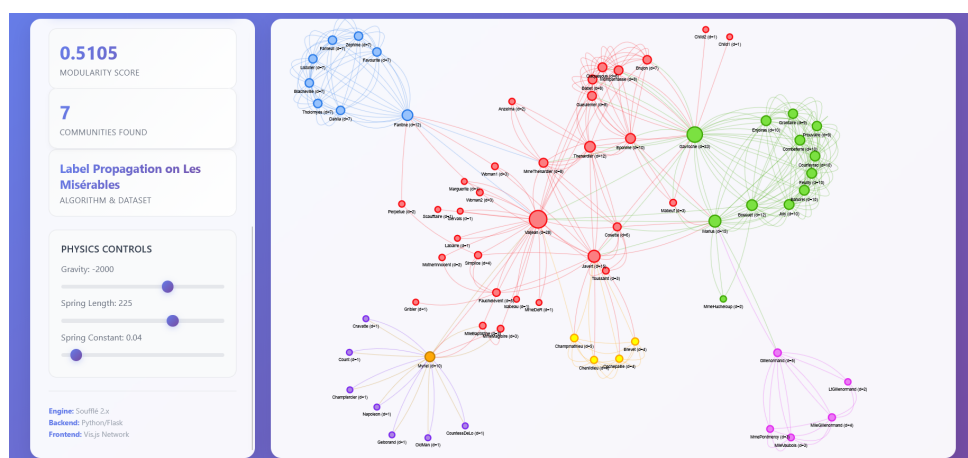


FIGURE 7.2 – Visualisation de l'algorithme de Propagation de Labels (LPA) en cours d'exécution.

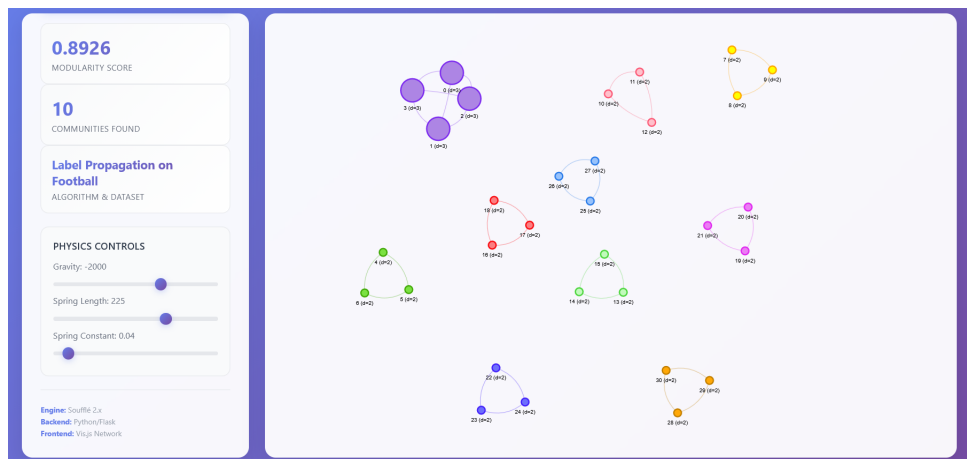


FIGURE 7.3 – Statistiques du réseau et calcul de la modularité via le moteur Soufflé.

Chapitre 8

Discussion et Perspectives

8.1 Avantages du Paradigme Déclaratif

L'utilisation de Datalog a permis de réduire le code "métier" à moins de 100 lignes tout en bénéficiant d'une parallélisation native. La séparation entre la logique (*Quoi*) et l'exécution (*Comment*) facilite grandement la maintenance et l'évolution des algorithmes.

8.2 Limites Rencontrées

La principale difficulté réside dans le caractère déterministe de Datalog. Là où les versions impératives de LPA utilisent l'aléa pour résoudre les égalités de labels, nous avons dû implémenter des règles de rupture de symétrie lexicographiques, ce qui peut légèrement influencer les résultats sur des graphes très réguliers.

8.3 Perspectives Futures

L'extension de ce travail pourrait inclure l'intégration d'algorithmes de détection de communautés chevauchantes (overlapping) ou l'utilisation de **Differential Datalog** pour traiter des flux de données en temps réel.

Chapitre 9

Évaluation et Comparaison des Paradigmes

Cette section compare l'approche déclarative (Datalog) avec l'approche impérative (Python/NetworkX) utilisée traditionnellement.

9.1 Tableau Comparatif des Paradigmes

Critère	Approche Datalog	Approche Impérative
Style	Déclaratif (Quoi)	Procédural (Comment)
Code Métier	< 100 lignes	> 300 lignes
Parallélisme	Natif (Automatique)	Manuel (Complexe)
Maintenance	Haute (Règles isolées)	Moyenne (Logique entrelacée)
Récursion	Native et optimisée	Réursive ou itérative manuelle

TABLE 9.1 – Comparaison entre Datalog et le paradigme impératif

9.2 L'Expressivité du Code

L'implémentation de la fermeture transitive ou de la propagation de labels en Datalog tient en moins de 10 lignes de logique pure. En comparaison, une implémentation Python équivalente nécessite la gestion de boucles, de conditions d'arrêt et de structures de données pour stocker les états intermédiaires, ce qui augmente le risque d'erreurs d'implémentation.

Chapitre 10

Conclusion

Ce projet a démontré la puissance de Datalog pour l'analyse de réseaux sociaux à grande échelle. En combinant la rigueur de la programmation logique avec la flexibilité du Web moderne, nous avons créé un outil performant, lisible et extensible pour la détection de communautés.

Chapitre 11

Codes Sources Datalog

Cette annexe présente les règles cœurs utilisées pour les différents algorithmes.

11.1 Label Propagation (lpa_step.dl)

```
1 .decl next_label(n: symbol, l: symbol)
2 next_label(N, L) :-
3     candidate_labels(N, L),
4     !smaller_candidate_exists(N, L).
```

11.2 Modularity (modularity.dl)

```
1 comm_contribution(C, Val) :-
2     community_edges(C, Lc),
3     community_degree_sum(C, Dc),
4     total_edges(M),
5     Val = (Lc / M) - (Dc / (2.0 * M))^2.0.
```

Bibliographie

- [1] U. N. Raghavan, R. Albert, and S. Kumara, *Near linear time algorithm to detect community structures in large-scale networks*, Physical Review E, vol. 76, no. 3, p. 036106, 2007.
- [2] H. Jordan, B. Scholz, and P. Subotić, *Soufflé : On Synthesis of Program Analyzers*, International Conference on Computer Aided Verification (CAV), pp. 422-430, Springer, 2016.
- [3] M. E. J. Newman, *Modularity and community structure in networks*, Proceedings of the National Academy of Sciences (PNAS), vol. 103, no. 23, pp. 8577-8582, 2006.
- [4] Vis.js Community, *Vis.js - A dynamic, browser based visualization library*, [En ligne]. Disponible sur : <https://visjs.org/>.
- [5] Pallets Projects, *Flask : A Python-based microframework for web development*, [En ligne]. Disponible sur : <https://flask.palletsprojects.com/>.